UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

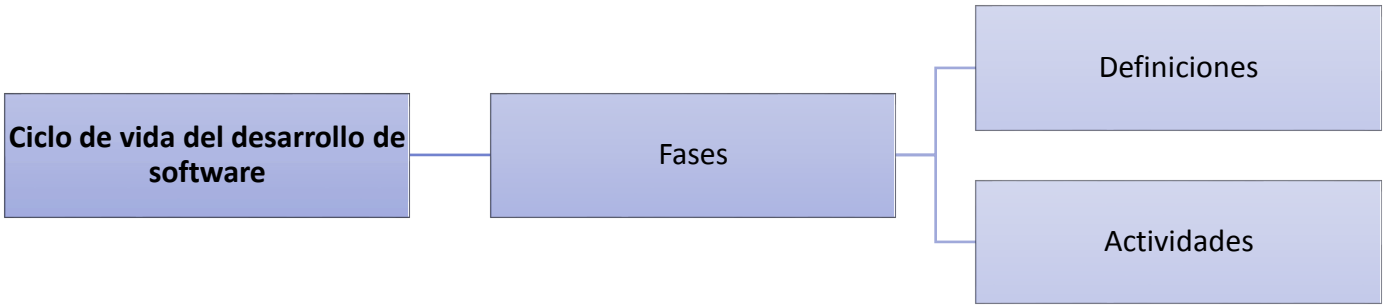
ANÁLISIS Y DISEÑO DE SISTEMAS

Área de Conocimiento | Ciencia y Tecnología

TABLA DE CONTENIDOS

Esquema 3
Breve Descripción del Capítulo 4
Objetivos 4
Ciclo de vida de los sistemas de información 5
1.1. Identificación del problema y factibilidad 5
1.1.1. Estudios de factibilidad 6
1.1.2. Análisis Costo-Beneficio 7
1.2. Análisis de requerimientos 10
1.2.1. Definiciones de Requisitos 10
1.2.2. Tipos de Requisitos 11
1.2.3. Especificación de Requisitos 11
1.3. Diseño lógico y físico 14
1.3.1. Diseño Lógico 14
1.3.2. Diseño Físico 15
1.4. Desarrollo y Documentación 16
1.4.1. Adquisición de hardware, sistemas operativos, bases de datos y telecomunicaciones 16
1.4.2. Codificación 17
1.4.3. Adquisición o desarrollo de software 18
1.4.4. Documentación 19
1.5. Pruebas del Sistema 20
1.6 Implementación 21
1.5.1. Capacitación del usuario 22
1.5.2. Instalación 22
1.5.3. Arranque 22
1.6. Producción y mantenimiento 23
1.6.1. Producción 23
1.6.2. Mantenimiento 24
1.7 Evaluación del Sistema 25
1.7.1 Auditoría Informática 25
1.7.2 Controles Internos 26
Bibliografía 27

Esquema


Ciclo de vida del desarrollo de software
Fases
Definiciones
Actividades

Breve Descripción del Capítulo

El ciclo de vida de desarrollo de software permite que el equipo desarrollador, el cliente y los usuarios puedan establecer los requisitos que deberá cumplir el sistema informático desarrollado. Cada fase tiene actividades que permiten afinar la solución planteada, revisando la factibilidad de construcción, desarrollando modelos y plasmando en un lenguaje de programación.

A la par de estas fases es necesario realizar pruebas antes, durante y después de la construcción del software.

La vida del software traspasa el tiempo de desarrollo y es importante conocer las actividades posteriores que deben realizarse cuando el sistema se encuentra en producción.

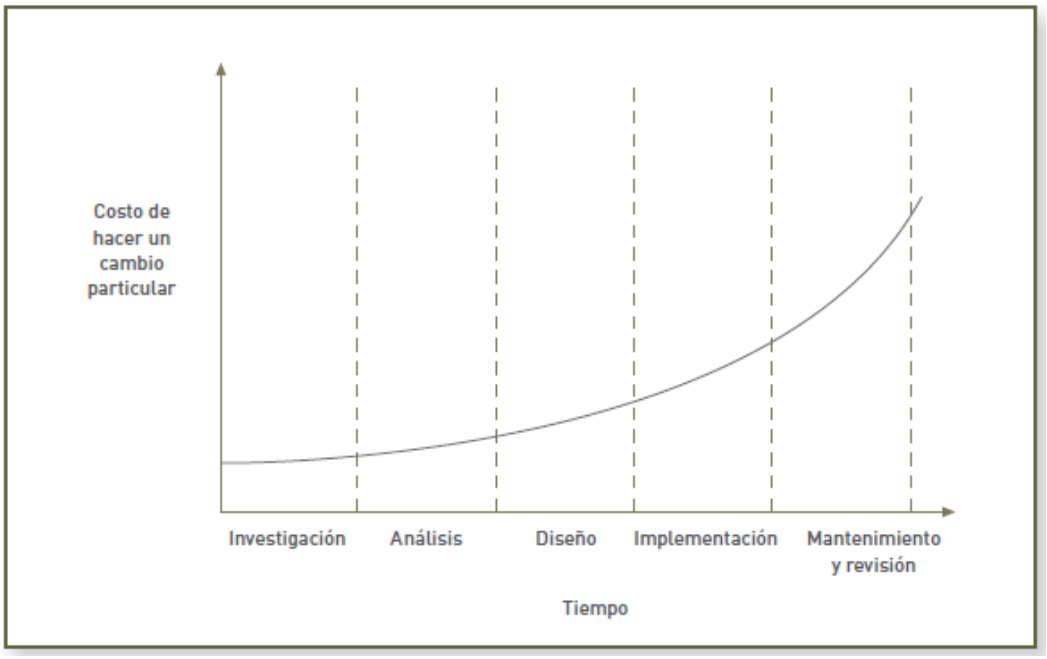
Objetivos

1. Identificar los procesos involucrados
2. Identificar las fases del desarrollo de un sistema de información

Ciclo de vida de los sistemas de información

Un proceso de software es una serie de actividades relacionadas que conduce a la elaboración de un producto de software. Estas actividades pueden incluir el desarrollo de software desde pero en un lenguaje de programación estándar. Sin embargo, las aplicaciones de negocios no se desarrollan precisamente de esta forma. El nuevo software empresarial con frecuencia ahora se desarrolla extendiendo y modificando los sistemas existentes, o configurando e integrando el software comercial o componentes del sistema. (Sommerville, 2011)

Existen muchos diferentes procesos de software, pero todos deben incluir cuatro actividades que son fundamentales para la ingeniería de software:
1. Especificación del software Tienen que definirse tanto la funcionalidad del software como las restricciones de su operación.
2. Diseño e implementación del software Debe desarrollarse el software para cumplir con las especificaciones.
3. Validación del software Hay que validar el software para asegurarse de que cumple lo que el cliente quiere.
4. Evolución del software El software tiene que evolucionar para satisfacer las necesidades cambiantes del cliente.



GRÁFICA 1: COSTO DE REALIZAR UN CAMBIO EN UN SISTEMA DE INFORMACIÓN FUENTE: (STAIR & REYNOLDS, 2010)

1.1. Identificación del problema y factibilidad

El objetivo global del desarrollo de sistemas es lograr metas empresariales, no metas técnicas, al entregar la información correcta a la persona apropiada en el momento oportuno. El impacto que un sistema particular tiene sobre la capacidad de una organización para satisfacer sus metas determina el verdadero valor de dicho sistema para la organización. (Alonso Amo, 2005)

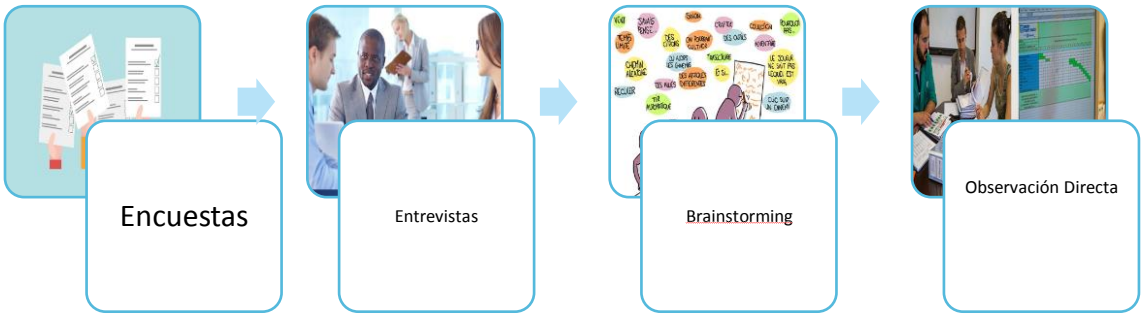
Ciclo de vida de los sistemas de información

El análisis del sistema de información de una compañía pequeña puede ser bastante directo, pero evaluar un sistema de información existente para una gran compañía puede ser un proceso largo y tedioso. Como resultado, las organizaciones grandes que evalúan un gran sistema de información por lo general siguen un procedimiento de análisis formalizado, que involucra los siguientes pasos:

1. Reunir a los participantes para el análisis de sistemas.
2. Recolectar datos y requerimientos adecuados.
3. Analizar los datos y requerimientos.
4. Preparar un reporte acerca del sistema existente, los requerimientos del sistema nuevo y las prioridades del proyecto. (Stair & Reynolds, 2010)

El analista, junto con el cliente, experto y usuarios, define los objetivos del sistema (el producto a obtener), la información que se va a suministrar al sistema y la que éste va a proporcionar, las funciones y el rendimiento requerido. Se puede usar varias técnicas para recopilar información de una organización como:

o



GRÁFICA 2: RECOPILAR INFORMACIÓN DE UNA ORGANIZACIÓN FUENTE: PRIETO P., 2020

ENCUESTA

Una encuesta puede capturar las opiniones de quienes no respondieron la entrevista.

ENTREVISTA

Las entrevistas pueden ser o estructuradas o abiertas. En una entrevista estructurada las preguntas se escriben por adelantado. En una entrevista abierta no se escriben por adelantado y el entrevistador se apoya en la experiencia al plantear las mejores interrogantes para descubrir los problemas inherentes al sistema existente. Una ventaja de la entrevista abierta es que permite al entrevistador plantear inmediatamente preguntas de seguimiento o aclaración.

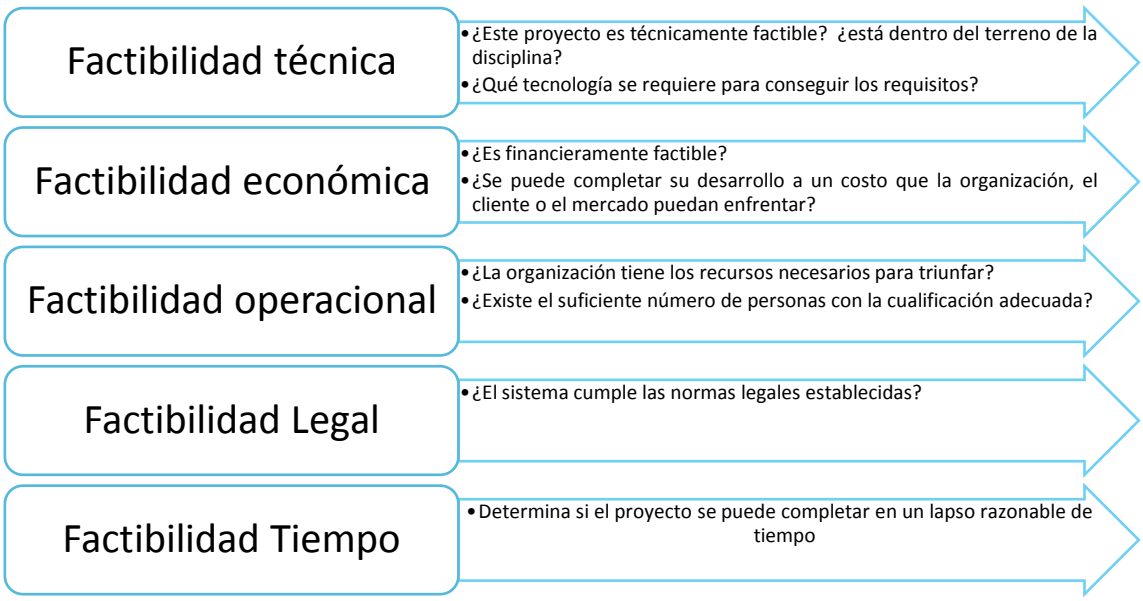
OBSERVACION DIRECTA

Con la observación directa, uno o más miembros del equipo de análisis observan directamente el sistema existente en acción. Una de las mejores formas para comprender cómo funciona éste es trabajar con los usuarios para descubrir cómo fluyen los datos en ciertas tareas empresariales.

Determinar el flujo de datos implica la observación directa de los procedimientos operativos de los usuarios, sus reportes, pantallas actuales (si ya está automatizado), etc. A partir de esta observación, los miembros del equipo de análisis determinan cuáles formatos y procedimientos son adecuados y cuáles son inadecuados y necesitan

1.1.1. Estudios de factibilidad

Los estudios de factibilidad deben realizarse antes de construir el sistema de información. Realizar este paso previo permite evaluar si es posible desde el punto de vista técnico, económico, operacional, legal o de tiempo desarrollar el sistema. Cuando el cliente o los ingenieros de software omiten esta etapa puede ser que un proyecto fracase.



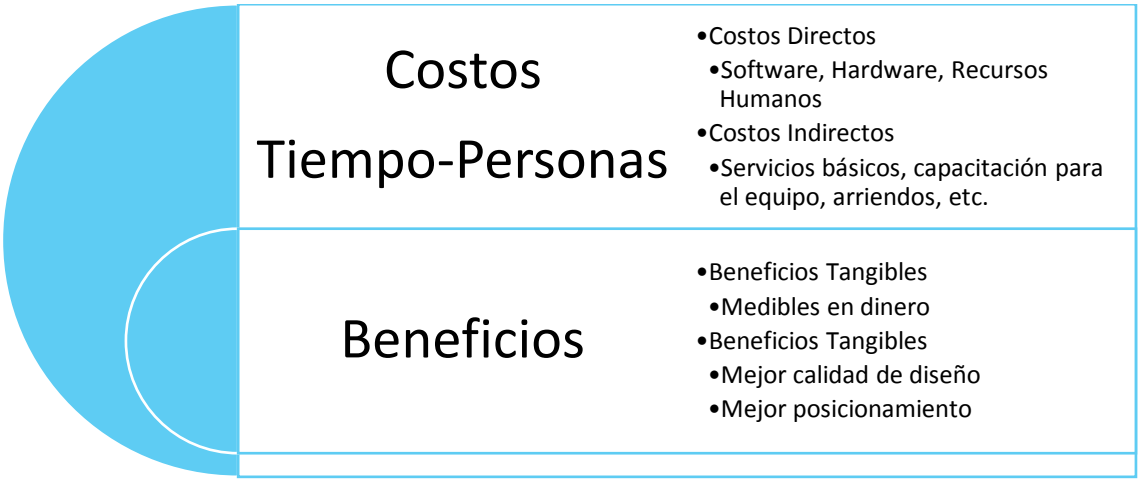
GRÁFICA 3: TIPOS DE FACTIBILIDAD QUE DEBE CONSIDERARSE EN LA PLANIFICACIÓN DE UN PROYECTO
FUENTE: (PRESSMAN, 2006)

1.1.2. Análisis Costo-Beneficio

El análisis del costo-beneficio es un proceso que, de manera general, se refiere a la evaluación de un determinado proyecto, de un esquema para tomar decisiones de cualquier tipo. Ello involucra, de manera explícita o implícita, determinar el total de costos y beneficios de todas las alternativas para seleccionar la mejor o más rentable. (Aguilera, 2017)

Es importante señalar que tomar una decisión implica elegir entre dos o más cursos de acción alternativos, por lo que el costo de oportunidad es otro factor a tener en cuenta, pues representa lo que se deja de ganar por haber rechazado el valor de la siguiente mejor opción. Siguiendo esta lógica, uno de los preceptos que propone el análisis costo-beneficio consiste en que no importa que tan adecuada sea la solución otorgada a un problema, la alternativa, o la propuesta, pues no dejará de tener un costo. En tal sentido, algunas cuestiones clave en el análisis serían:

- Si el costo de la solución sobrepasa el del problema.
- Si la solución es más cara pero trae mejorías que no se cuantifican en términos monetarios e influyen en el aspecto social.
- ¿Se debe considerar aquella información que afecta los posibles cursos de acción? (Aguilera, 2017)



GRÁFICA 4: ANÁLISIS COSTO - BENEFICIO FUENTE: PRIETO P., 2020

COSTOS

Es útil distinguir entre dos tipos de costes asociados con los proyectos de sistemas de información: tangibles o visibles e intangibles u ocultos. Los costes tangibles son denominados, con frecuencia, visibles dado que se pueden medir de una forma razonablemente inmediata. Los costes intangibles se denominan, con frecuencia, ocultos dado que la mayoría de las organizaciones experimenta dificultades en la cuantificación de tales costes. (Beynon-Davies, 2014)

A continuación se muestra una lista típica de tipos de coste asociados a los proyectos de sistemas de información:

- Costes hardware: computadores, impresoras, almacenamiento, accesorios, etc.
- Costes software: paquetes llave en mano, software a medida, herramientas de desarrollo, etc.
- Costes de instalación: introducción de datos, conversión de datos, etc.
- Costes ambientales: cableado, mobiliario, aire acondicionado, etc.
- Costes de operación: electricidad, comunicaciones, etc.
- Costes de mantenimiento: del hardware y software.
- Costes de seguridad: gestión del riesgo y mecanismos de recuperación de desastres. Costes de red: hardware, software y mantenimiento de la red.
- Costes de formación: con frecuencia subestimados.
- Costes generales de la organización: nuevas estructuras salariales, dirección, etc. (Beynon-Davies, 2014)

BENEFICIOS

Los beneficios consideran el valor que la organización adquiere por tener un sistema de información. Nuevamente, podemos distinguir entre beneficios tangibles e intangibles. Con frecuencia, los sistemas de información han sido iniciados con el objetivo de obtener beneficios como la reducción de plantilla o el aumento de productividad.

Los beneficios intangibles se asocian más generalmente a aspectos de eficiencia de la organización. Más recientemente, se ha comenzado a debatir si los beneficios intangibles como el aumento de la satisfacción del cliente o la construcción de mejores vínculos con los

suministradores tienen la misma relevancia en las decisiones sobre inversiones. Los beneficios intangibles se asocian más generalmente con la efectividad de la organización. A continuación enumeramos algunos de los beneficios clásicos asociados a los sistemas de información:

- Precisión : aumento de la precisión de la información
- Calidad : información de mayor calidad
- Usabilidad : información más útil
- Flexibilidad : capacidad de utilizar la información de forma más flexible
- Satisfacción del usuario : mayor satisfacción laboral
- Funcionalidad : trabajo más eficaz
- Fiabilidad : servicio más fiable Utilización : más información utilizada
- Relevancia : información mejor enfocada
- Productividad: mayores niveles de productividad (Beynon-Davies, 2014)

El análisis coste-beneficio resulta fundamental en la evaluación de si merece la pena invertir en el proceso de desarrollo de un sistema de información. La mayor parte de las técnicas establecidas para la evaluación de inversiones de sistemas de información se concentran en los costes y beneficios tangibles y, por ello, se dirigen primordialmente a la evaluación de las mejoras de la eficiencia. Dos de las más populares son el retorno de la inversión y el periodo de retorno. (Beynon-Davies, 2014)

RETORNO DE LA INVERSION - ROI

Esta medida establece las ganancias o beneficios adicionales generados, y se expresa como un porcentaje de la inversión en tecnología de sistemas de información. (Stair & Reynolds, 2010)

Así, para calcular el ROI uno debe ser capaz de estimar los ingresos acumulados por la introducción de un sistema de información y el coste asociado al sistema de información para un periodo del futuro. El promedio de esta relación entre los costes anuales y los beneficios es la tomada para indicar el valor del sistema de información para la organización. (Beynon-Davies, 2014)

PERIODO DE RETORNO

El periodo de retorno aún asume que uno es capaz de estimar el beneficio de la introducción de un sistema de información para la organización a varios años vista. El beneficio se mide en función de los flujos de entrada de caja resultantes del sistema de información. El periodo de retorno es igual al número de meses o años necesarios para que la cifra del retorno se haga cero. (Beynon-Davies, 2014)

Retorno de la inversión

- $RoI = promedio\left(\frac{ingresos\ anuales\ netos}{inversion\ anual}\right)$

Periodo de Retorno

- $Retorno = inversion\ - beneficio\ acumulado(flujos\ entrantes\ en\ caja)$

GRÁFICA 5: FÓRMULAS PARA EVALUACIÓN DE COSTO-BENEFICIO FUENTE: (BEYNON-DAVIES, 2014)

1.2. Análisis de requerimientos

Esta etapa tiene por objeto realizar un análisis global del sistema, estableciendo los requisitos de todos los elementos del sistema y luego (debido a que el software es por lo general una parte del sistema) asignar al software la parte de los requisitos que le afectan. Este planteamiento del sistema es esencial cuando el software debe interrelacionarse con otros elementos, tales como personas, hardware y bases de datos.

Un requisito es una característica deseada de un sistema de información. Los requisitos se describen tradicionalmente como no problemáticos en el sentido de que pueden ser obtenidos sin demasiados inconvenientes por parte de la comunidad de grupos de interés. Sin embargo, los requisitos tienen las siguientes características:

• Los requisitos pueden variar dependiendo del grupo de interés. Los requisitos no son fenómenos objetivos, están relacionados con la perspectiva de un grupo de interés particular.

• Los requisitos pueden estar enfrentados. El análisis conlleva intentar conseguir un cierto acuerdo intersubjetivo en relación con los requisitos entre los distintos grupos de interés.

• Los requisitos deben paralizarse en algún punto para construir un sistema de tecnología de información, un artefacto. Sin embargo, es probable que los requisitos cambien con el tiempo. (Beynon-Davies, 2014)

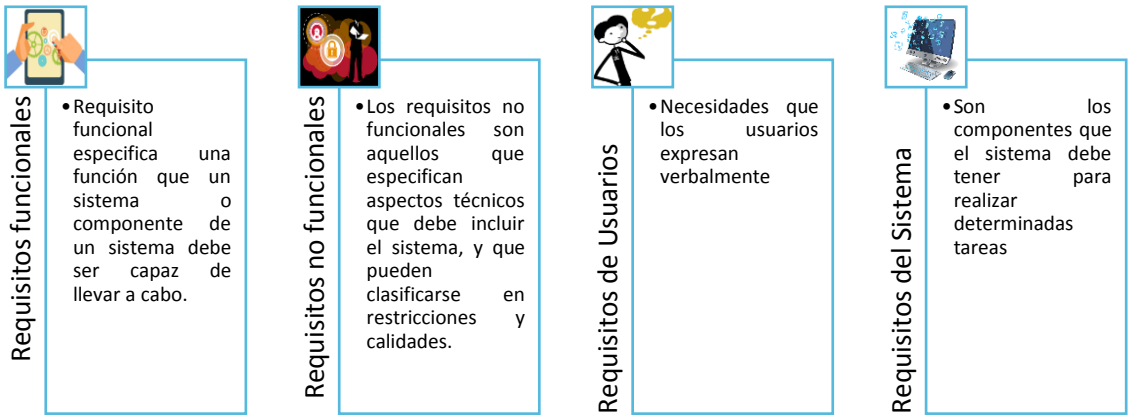
1.2.1. Definiciones de Requisitos

Un requisito de software es la capacidad que debe alcanzar o poseer un sistema o componente de un sistema para satisfacer un contrato, estándar, especificación u otro documento formal. (IEEE)

Un requisito de software es la propiedad que un software desarrollado o adaptado debe tener para resolver un problema concreto. (SWEBOK)

La verificabilidad es una propiedad intrínseca de un requisito, por la cual es posible comprobar que el sistema software en su conjunto, o alguno de sus componentes, satisfacen el requisito tal y como fue especificado.

1.2.2. Tipos de Requisitos



GRÁFICA 6: TIPOS DE REQUISITOS FUENTE: PRIETO P., 2020

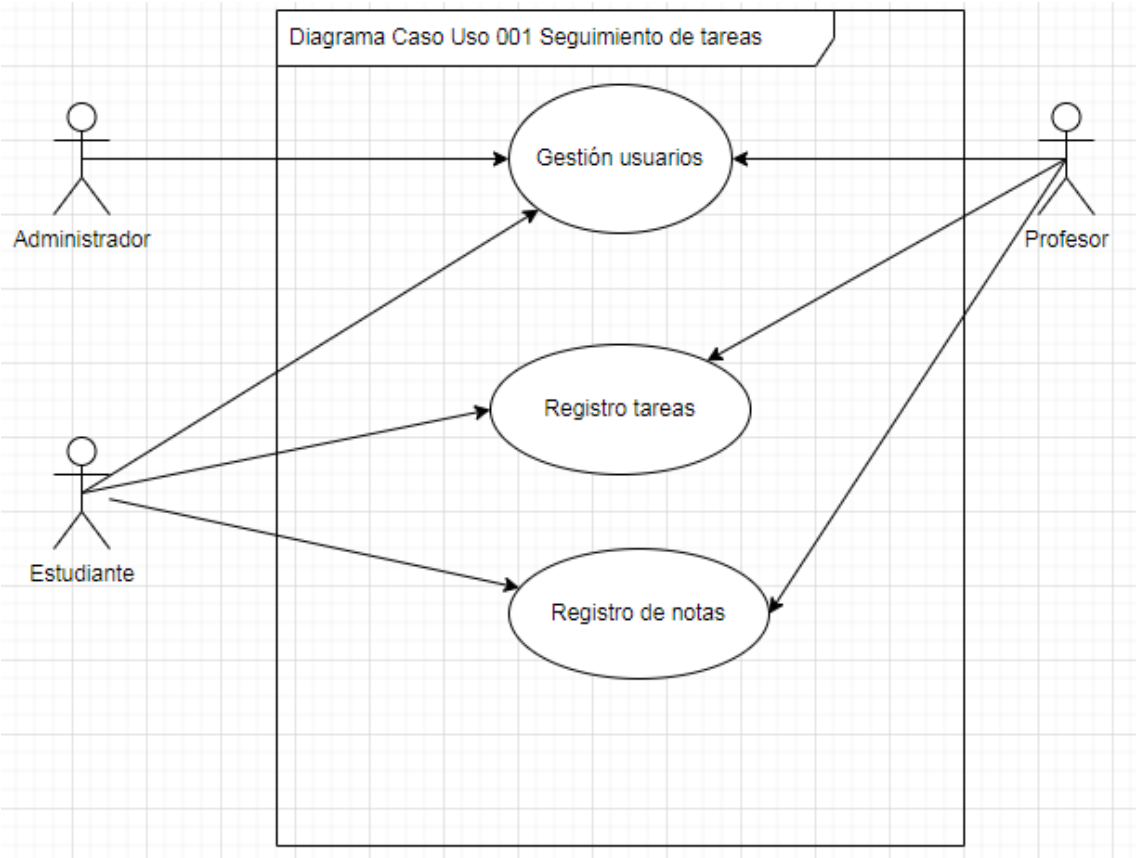
1.2.3. Especificación de Requisitos

La especificación de requisitos es el proceso relativo a la representación de los requisitos establecidos en el proceso de obtención de requisitos. Tradicionalmente la especificación de requisitos conlleva alguna forma de representación intermedia, determinada notación para representar requisitos además de su estructura en cuanto a determinado sistema. La notación utilizada es frecuentemente gráfica, en ocasiones textual y a veces matemática. En un contrato formal es necesario anexar la especificación de requisitos de software. (Beynon-Davies, 2014)

La especificación de requisitos de software es una descripción completa del comportamiento del sistema que se va a desarrollar. Incluye un conjunto de casos de uso que describe todas las interacciones que tendrán los usuarios con el software. Los casos de uso también son conocidos como requisitos funcionales.

CASOS DE USO

Un modelo de caso de uso proporciona una descripción de alto nivel de las principales interacciones del usuario con un sistema de información. Tal modelo utiliza dos conceptos: actores y casos de uso. Un actor es cualquier persona, organización o sistema que interactúa con determinado sistema de información. Un caso de uso es un conjunto delimitado de actividades que podemos especificar como elementos importantes de la funcionalidad de determinado sistema de información. (Beynon-Davies, 2014)



GRÁFICA 7: CASO DE USO FUENTE: PRIETO P., 2020

PLANTILLA ESTÁNDAR

Se suele usar una plantilla estándar para que los requisitos sean presentados de una manera más consistente y por ende más entendible. El documento escrito debe combinar descripciones en el lenguaje natural y modelos gráficos. (Pressman, 2006)

Actualmente se encuentran en uso las siguientes plantillas estándares propuestas por la IEEE- Institute of Electrical and Electronics Engineers e ISO- International Organization for Standardization.

IEEE 830-1998

ISO/IEC/IEEE 29148:2011 Systems and software engineering — Life cycle processes — Requirements engineering

ISO/IEC/IEEE 29148:2018(en) Systems and software engineering — Life cycle processes — Requirements engineering

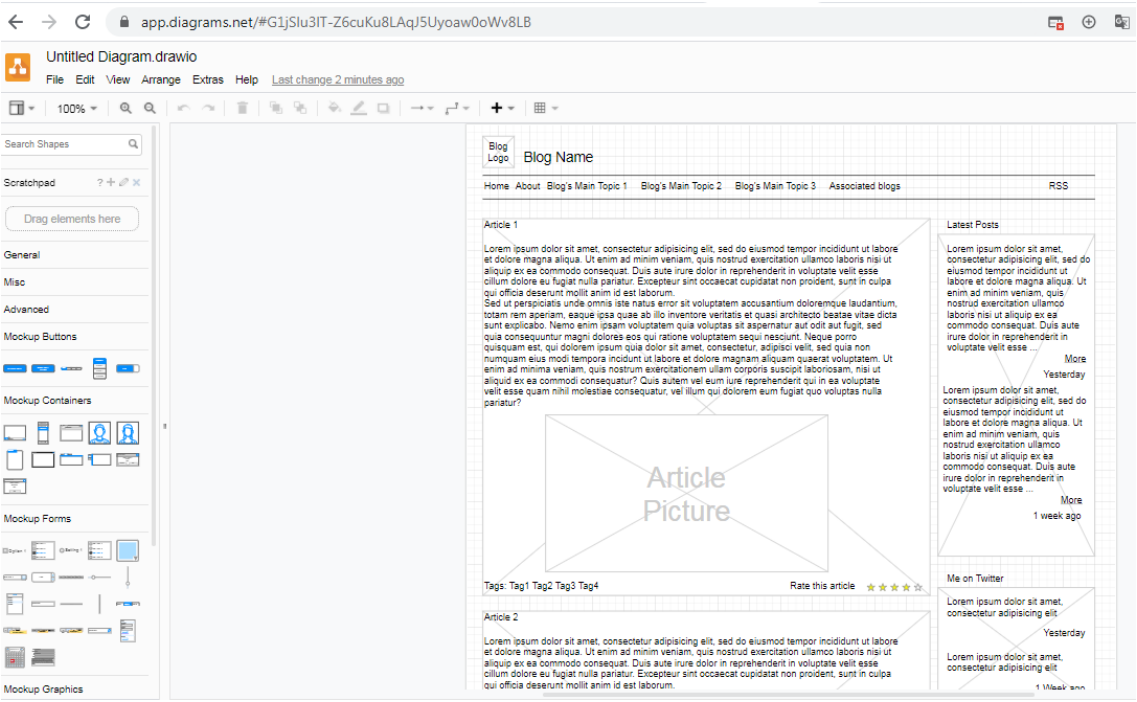
GRÁFICA 8: PLANTILLAS ESTÁNDAR PARA LA ESPECIFICACIÓN DE REQUERIMIENTOS **FUENTE:** PRIETO P., 2022

PROTOTIPADO

Es una versión del sistema o una parte del mismo, que se crea para comprobar los requerimientos del cliente y la factibilidad de algunas decisiones de diseño.

Esto apoya el hecho de evitar el cambio, al permitir que los usuarios experimenten con el sistema antes de entregarlo y así refinar sus requerimientos. Como resultado, es probable que se reduzca el número de propuestas de cambio de requerimientos posterior a la entrega.

Existen varias herramientas CASE que permiten crear prototipos entre ellas se encuentran: Pencil Project, Drawio, Mockup Screen, etc.



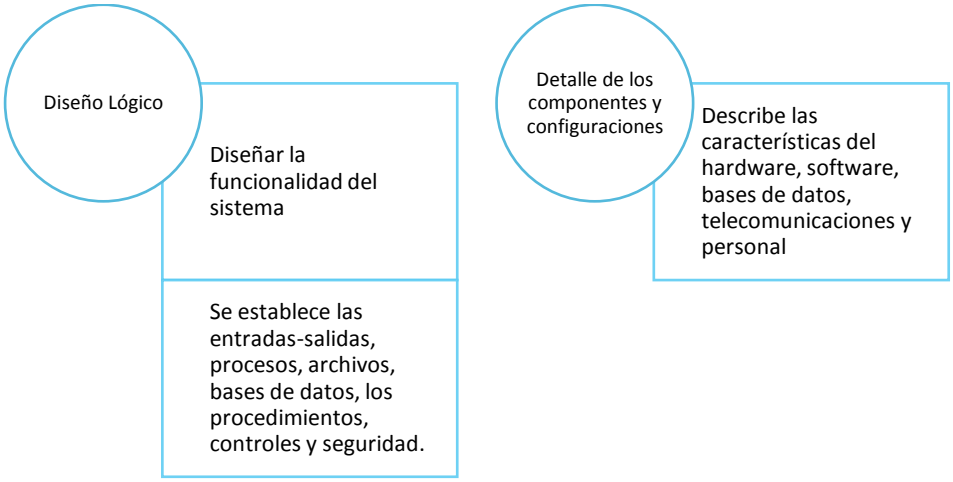
GRÁFICA 9: USO DE DRAWIO PARA ELABORAR UN PROTOTIPO **FUENTE:** APP.DIAGRAMS.NET., 2022

1.3. Diseño lógico y físico

El diseño es el primer paso en la fase de desarrollo de cualquier producto o sistema de ingeniería. Se define como "el proceso de aplicar distintas técnicas y principios con el propósito de definir un dispositivo, proceso o sistema con los suficientes detalles como para permitir su realización física".

El objetivo del diseñador es producir un modelo o representación de una entidad que será construida más adelante.

El propósito del diseño de sistemas es responder la pregunta: ¿cómo el sistema de información resolverá un problema? El principal resultado de la fase de diseño de sistemas es un producto técnico que detalla las salidas, entradas e interfaces del usuario del sistema; especifica hardware, software, bases de datos, telecomunicaciones, personal y procedimientos, y muestra cómo se relacionan dichos componentes. El sistema nuevo o modificado debe sacar ventaja de los más recientes desarrollos en tecnología. (Stair & Reynolds, 2010)



Gráfica 10: DESCRIPCIÓN DEL DISEÑO LÓGICO Y DISEÑO FÍSICO **FUENTE:** (Stair & Reynolds, 2010)

1.3.1. Diseño Lógico

El diseño lógico involucra la planeación del propósito de cada elemento del sistema, independientemente de consideraciones de hardware y software. Las especificaciones del diseño lógico que se determinan y documentan incluyen salida, entrada, proceso, archivo y base de datos, telecomunicaciones, procedimientos, controles y seguridad, y requerimientos de personal y de trabajo.

Diseño de entradas, diseño de salidas	Diseño de procesos	Diseño de datos	Diseño de comunicación	Diseño de controles y seguridad
• Diseño de las entradas y salidas del sistema de información • Modelo Contexto • Diagrama de Flujo de datos	• Diseño del funcionamiento de la organización	• Identificar todas las estructuras de datos y operaciones sobre ellas. • Crear un modelo lógico de la base de datos • Establecer un diccionario de datos	• El diseño marco de las comunicaciones	• Diseño de la gestión de desastres y de seguridad

Gráfica 11: DETALLE DEL DISEÑO LÓGICO **FUENTE:** (Stair & Reynolds, 2010)

1.3.2. Diseño Físico

El diseño físico, por su parte, se refiere a cómo se logran las tareas, incluido cómo los componentes trabajan en conjunto y qué hace cada uno. El diseño físico especifica las características de los componentes de sistema necesarios para poner en acción el diseño lógico. En esta fase deben describirse con detalle las características del hardware, software, bases de datos, telecomunicaciones y personal, así como las especificaciones de procedimiento y control.

Diseño Hardware	Diseño Software	Diseño de las comunicaciones	Diseño de bases de datos	Diseño de interfaces
• Detallar los dispositivos hardware que se requiere	• Desarrollar una estructura modular del software y representar las relaciones de control entre los módulos	• Detallar las formas de comunicación entre el hardware y software	• Elaborar un modelo físico de la base de datos • Implementar el modelo en un gestor de bases de datos	• Muestra la información que es relevante para el contexto • Produce mensajes de error significativos • El significado de una pantalla debe ser obvio

Gráfica 12: DETALLE DEL DISEÑO FÍSICO **FUENTE:** (Stair & Reynolds, 2010)

1.4. Desarrollo y Documentación

Esta etapa tiene por fin traducir en una forma legible para la computadora el diseño desarrollado en la etapa anterior. Esta actividad implica la creación de programas informáticos aplicando las estructuras de programación de algún paradigma y utilizando un lenguaje apropiado de programación. Además este proceso, incluye adquisición de hardware, codificación y adquisición o desarrollo de software y capacitación de usuarios

1.4.1. Adquisición de hardware, sistemas operativos, bases de datos y telecomunicaciones

Para obtener los componentes de un sistema de información, las organizaciones pueden comprar, arrendar (leasing) o rentar hardware de cómputo y otros recursos de un proveedor. Éste es una compañía que ofrece hardware, software, sistemas de telecomunicaciones, bases de datos, personal si y otros recursos relacionados con computadoras

En la adquisición de hardware también se incluye la compra de software para sistemas operativos, gestores de bases de datos e incluso se debe considerar las cuestiones sobre telecomunicaciones

Gráfica 13: ELEMENTOS NECESARIOS EN LA IMPLEMENTACION DE UN SISTEMA DE INFORMACION
FUENTE: PRIETO P., 2020

1.4.2. Codificación

La codificación de un sistema consiste en la traducción del diseño de dicho sistema a un lenguaje de programación. En algunos casos esta traducción se hace utilizando herramientas case y en todos los casos usando el talento de los programadores que elaboran dicho código. Como producto de esta tarea se obtiene un "listado fuente" de los programas que definen el software que se está desarrollando.

El lenguaje de programación se escoge en función de la solución planteada pues existen lenguajes que permiten la creación de sitios web, aplicaciones para sistemas embebidos, aplicaciones móviles, sistemas operativos, etc. En la figura siguiente se muestran los lenguajes de programación más usados en la actualidad según el índice TIOBE https://www.tiobe.com/tiobe-index/

Mar 2022	Mar 2021	Change	Programming Language	Ratings	Change
1	3	^	Python	14.26%	+3.95%
2	1	v	C	13.06%	-2.27%
3	2	v	Java	11.19%	+0.74%
4	4		C++	8.66%	+2.14%
5	5		C#	5.92%	+0.95%
6	6		Visual Basic	5.77%	+0.91%
7	7		JavaScript	2.09%	-0.03%
8	8		PHP	1.92%	-0.15%
9	9		Assembly language	1.90%	-0.07%
10	10		SQL	1.85%	-0.02%

Gráfica 14: LISTADO DE LENGUAJES DE PROGRAMACIÓN MÁS USADOS **FUENTE:** https://www.tiobe.com/tiobe-index/, 2022

Para lograr que el código fuente sea fácil de mantener y adaptar a lo largo del ciclo de vida se recomienda usar estándares en la codificación. En el gráfico siguiente se muestra varias pautas que pueden ser de utilidad para el desarrollo en un equipo de trabajo.

Código Fuente	Datos	Sentencias	Entradas/Salidas
•El código fuente debe documentarse con la elección de identificadores que sean significativos •Incluir comentarios en los módulos. •Usar indentado para realzar las construcciones lógicas y los bloques de código.	•Los datos deben declararse en lugares de fácil acceso •Se sugiere declarar los datos siguiendo este orden: •Declaraciones de tipo simple •Declaraciones de tipo estructurado •Declaraciones de archivos	•Las sentencias deben ser simples y fáciles de seguir. •Usar paréntesis para dar legibilidad a la sentencia. •No usar sentencias de iteración con muchos anidamientos. •Usar varias líneas para sentencias largas.	•Mantener un formato de entrada simple. •Validar los datos de entrada

Gráfica 15: BUENAS PRÁCTICAS DE CODIFICACIÓN **FUENTE:** (Sommerville, 2011)

1.4.3. Adquisición o desarrollo de software

Los analistas y las organizaciones se enfrentan cada vez más con la decisión de crear, comprar o subcontratar al evaluar software para los proyectos de sistemas de información. El software se puede comprar ya desarrollado. Los componentes de software se pueden adquirir y luego modificar e integrar para satisfacer las necesidades específicas. El software se puede construir de manera personalizada por medio de un contratista externo o de forma interna; para satisfacer las necesidades del cliente.

A continuación se muestra una tabla que muestra las ventajas y desventajas asociadas a la adquisición o desarrollo de software. Internamente la empresa debe decidir cuál es la opción más apropiada.

Adquisición o desarrollo	Definición	Ventajas	Desventajas
Creación de software personalizado	Cuando no existe un software comercial. Cuando la organización trata de lograr una ventaja competitiva	Respuestas específicas a las necesidades especializadas de la empresa. La innovación puede dar a la empresa una ventaja competitiva Personal interno disponible para dar mantenimiento al software.	Puede tener un costo inicial muy alto. Es necesario contratar un equipo de desarrollo. Mantenimiento continúo.
Comprar paquetes o software comercial	Cuando el software comercial puede integrarse fácilmente con las aplicaciones de la organización. Cuando no existe una necesidad inmediata de modificar el software	Mayor confiabilidad, funcionalidad. Tiene un menor costo inicial. Capacitación incluida en el software.	Enfocado en la programación, no en la empresa. Hay que acoplarse a las características existentes. Personalización limitada. Menor propiedad y compromiso.
Subcontratar servicios	Cuando la empresa desea dedicarse a su actividad económica.	Cuando una organización se enfoca en su misión estratégica. No se gasta el tiempo de los empleados en tareas de TI no esenciales	Pérdida del control de los datos, sistemas, empleados TI. Cuestiones de seguridad, confidencialidad y privacidad. Pérdida de ventaja corporativa

TABLA 1: VENTAJAS Y DESVENTAJAS DE LA ADQUISICIÓN O DESARROLLO DE SOFTWARE FUENTE: (PRESSMAN, 2006)

De forma adicional se puede mencionar la posibilidad de adquirir el software como un servicio (SaaS) ya que permite a las empresas suscribirse a software de aplicación entregado por web pagando un cargo mensual por servicio o una tarifa por uso. En lugar de adquirir software externamente de un proveedor tradicional, SaaS permite a los individuos y organizaciones acceder a las aplicaciones necesarias a través de internet. (Beynon-Davies, 2014)

1.4.4. Documentación

Con el software desarrollado internamente, la documentación siempre es importante. La documentación técnica la usan los operadores de las computadoras para ejecutar el programa, y los analistas y programadores para resolver problemas o modificar el programa. En la documentación técnica se escribe y explica el propósito de cada pieza de código de computadora. También se describen las variables clave. La documentación de usuario se desarrolla para las personas que usan el programa y muestra, en una forma fácil de entender, cómo se puede y debe usar el programa. Incorporar en la documentación de usuario una descripción de los beneficios de la nueva aplicación ayuda a los interesados a comprender las razones del programa y acelera la aceptación del usuario. (Beynon-Davies, 2014)

La documentación consiste en plasmar en un documento escrito o electrónico todas las especificaciones técnicas de diseño con el fin de que otros analistas y programadores entiendan cómo se diseñó y desarrolló y lo puedan mantener y mejorar.

MANUAL DE USUARIO

Un manual de usuario es un documento que aborda un aspecto específico de un producto de software. En algunos sistemas de software de negocios, donde los grupos de usuarios tienen acceso sólo a una de la funcionalidad total de la aplicación, es bastante aconsejable preparar una guía de usuario para cada grupo.

MANUAL DE CONFIGURACIÓN

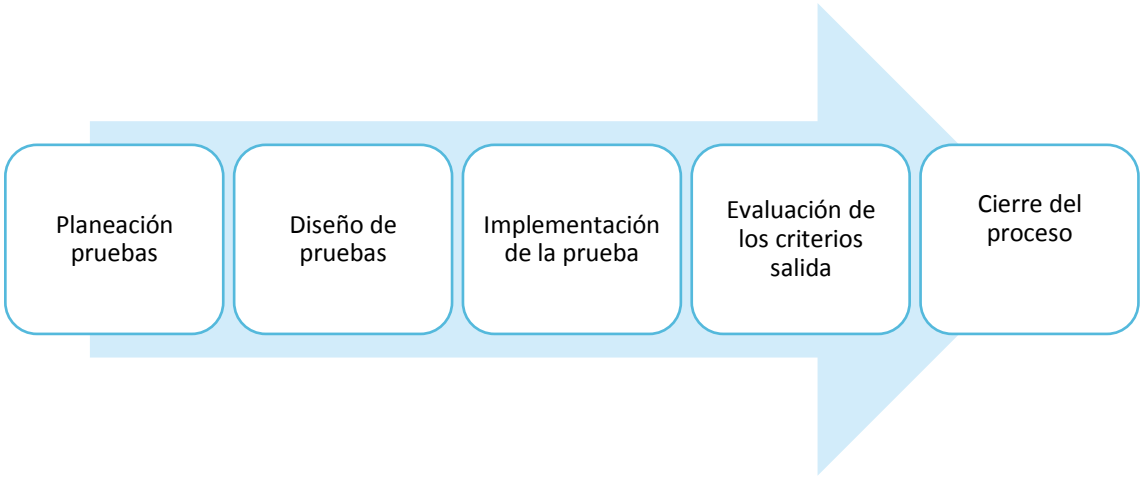
Este manual está orientado a un usuario técnico o administrador del sistema. Esta persona será la encargada de la instalación y el mantenimiento del sistema. Por tanto requiere información técnica específica que le permita realizar las siguientes tareas:

- Instalación del sistema informático.
- Creación de backup de datos y programas.
- Recuperación de la base de datos y/o programas.
- Migración de datos.
- Instalación y configuración de la base de datos.
    - Tipo de conexión (ODBC, Puerto, etc.)
    - Nombre de la base de datos
    - Nombre del usuario y clave por defecto
    - Configuración del idioma

- Instalación y configuración de los servidores.
    - Definir la arquitectura (cliente – servidor, tres capas, n capas, etc.)
    - Definir el tipo de red (intranet, extranet, vpn, internet, etc.)
    - En aplicaciones web definir si es necesario compra de dominio y compra o arrendamiento de hosting.

1.5. Pruebas del Sistema

Los buenos procedimientos de pruebas son esenciales para asegurarse de que el sistema de información nuevo o modificado opera como se pretende. Una prueba inadecuada puede resultar en errores y problemas.



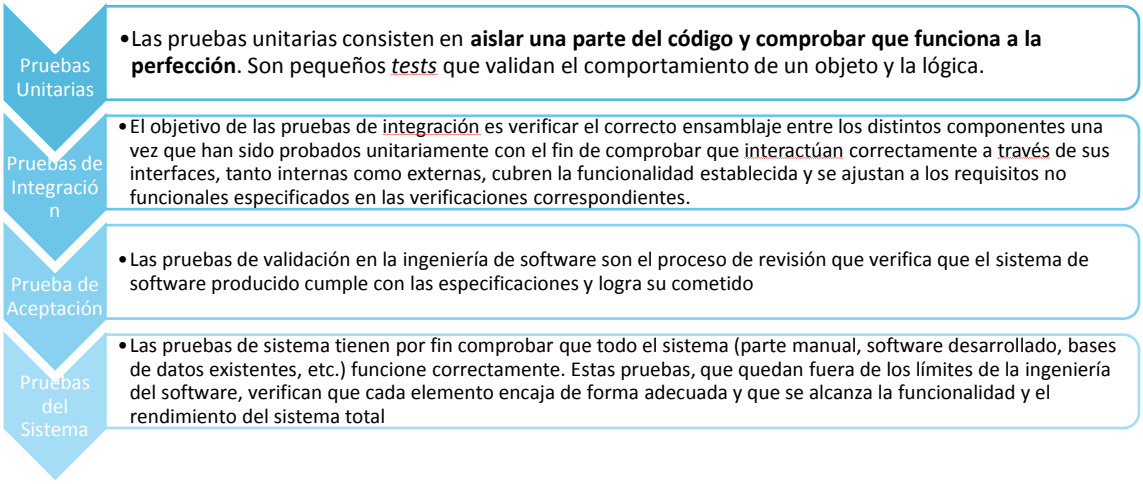
Gráfica 16: PASOS PARA IMPLEMENTAR EL PROCESO DE PRUEBAS **FUENTE:** (Sommerville, 2011)

Deben usarse varias formas de pruebas, incluidas la prueba de cada programa (prueba unitaria), de todos los programas del sistema (prueba del sistema), de la aplicación con una gran cantidad de datos (prueba de volumen) y del conjunto de todos los sistemas relacionados (prueba de integración), así como realizar cualquier otra requerida por el usuario (prueba de aceptación). (Stair & Reynolds, 2010)

Pruebas de desarrollo	Pruebas del sistema	Pruebas de aceptación
• Los programadores prueban el sistema • Cada componente se prueba de manera independiente. • JUnit	• Los componentes se integran para descubrir errores en la interacción. • Verificar que el sistema cumple con los requerimientos funcionales y no funcionales	• Se prueba con datos reales • Revisar si se cumplen los requerimientos del sistema

Gráfica 17: PRUEBAS EN UN SISTEMA DE INFORMACIÓN **FUENTE:** (Pressman, 2006)

Para una descripción más detallada sobre los tipos de pruebas se tiene los gráficos siguientes:



Gráfica 18: TIPOS DE PRUEBAS EN UN SISTEMA DE INFORMACIÓN **FUENTE:** (Pressman, 2006)

Pruebas de Volumen	Pruebas de Seguridad	Pruebas de Rendimiento	Pruebas de Usuario: alfa y beta
•Sobrecarga un sistema para encontrar el punto de ruptura.	•Comprueban que los mecanismos de protección integrados en el sistema realmente lo protejan de irrupciones inapropiadas	•Están diseñadas para probar el desempeño del software en tiempo de ejecución dentro del contexto de un sistema integrado.	•Al construir software personalizado para un cliente se aplica una serie de pruebas de aceptación que permita al cliente validar todos los requisitos.

Gráfica 19: TIPOS DE PRUEBAS DE SISTEMA **FUENTE:** (Pressman, 2006)

1.6 Implementación

El proceso de implementación de sistemas (en ocasiones denominado entrega de sistemas) sigue a la construcción de sistemas. La implementación de sistemas conlleva la entrega de un sistema de información a su contexto de uso. Dado que el sistema de información es un sistema socio-técnico, la implementación de tales sistemas conlleva la implementación paralela de un sistema de información y de algún tipo de sistema de actividad humana. Una vez que se entrega un sistema a su contexto de uso estará sujeto al proceso de operación y al proceso de mantenimiento del sistema. (Beynon-Davies, 2014)

En esta etapa se realiza la contratación y capacitación de personal, preparación de sitio y datos, instalación, prueba, arranque y aceptación de los usuarios.

1.5.1. Capacitación del usuario

La preparación del usuario es el proceso de capacitar en el nuevo sistema a administradores, a quienes toman decisiones, a empleados y a otros usuarios e interesados. Esta actividad es una área importante, aunque frecuentemente olvidada, de la implementación de sistemas. Cuando se implementa un nuevo sistema operativo o paquete de software de aplicación, la capacitación del usuario es esencial.

Se recomienda para esta etapa:

1. Crear medios de entrega
    1. Ensamblar y probar todos los archivos ejecutables.
    2. Ensamblar y probar todos los archivos de datos
    3. Crear y probar toda la documentación del usuario.
    4. Implementar archivos de ayuda
    5. Implementar una guía para la resolución de problemas.
2. Establecer la persona o grupo encargado del soporte humano
    1. Establecer mecanismos de contacto: sitio web, teléfono, email, etc.
    2. Establecer mecanismos para la localización de problemas.
    3. Establecer mecanismos para el reporte de problemas.
    4. Establecer una base de datos para el reporte de problemas.
3. Establecer mecanismos de retroalimentación del usuario
    1. Definir el proceso de retroalimentación.
    2. Definir las formas de retroalimentación
4. Diseminar los medios de entrega a todos los usuarios.
5. Dirigir las funciones de soporte continuas.
    1. Proporcionar asistencia en la instalación
    2. Proporcionar asistencia continua y de resolución de problemas.

1.5.2. Instalación

La instalación es el proceso de colocar físicamente el equipo de cómputo en el sitio o centro de datos y hacerlo operativo. Aunque comúnmente el fabricante es responsable de instalarlo, alguien en la organización (por lo general el gerente de tecnologías de la información) debe supervisar el proceso, y asegurarse de que todo el equipo especificado en el contrato se instale en la ubicación adecuada. Después de instalar el sistema, el fabricante realiza varias pruebas para asegurarse de que el equipo opera como es debido.

1.5.3. Arranque

El arranque comienza con el sistema de información ya sometido a las pruebas finales. Cuando se termina el procedimiento de arranque, el sistema es totalmente operativo. Existen diversos métodos de arranque disponibles, entre estos se tiene: arranque en paralelo, en fases, abrupta y mediante un piloto.

ARRANQUE EN PARALELO

Se utiliza el sistema antiguo junto con el nuevo durante un período predeterminado. Esta duplicidad reduce el riesgo causado por alguna falla del sistema nuevo, el inconveniente es que es costosa por la mano de obra al ejecutar los dos sistemas.

ARRANQUE POR FASES

Los sistemas de información sobre todo los grandes, se dividen en módulos funcionales, los cuales se ponen en operación uno a la vez, este proceso de llama conversión en fases. Los usuarios aprenden a emplear un módulo a la vez, lo cual es más fácil que aprender que el sistema completo. Sin embargo cuando se utilizan partes de ambos sistemas pueden ocurrir inconsistencia entre los datos de los dos.

ARRANQUE ABRUPTO

Se la llama también conversión de corte rápido consiste en descartar el sistema antiguo y el nuevo se encarga de toda la operación empresarial.

Esta estrategia es muy arriesgada, pero si tiene éxito su costo es muy bajo, porque no se gastan recursos en ejecutar dos sistemas en paralelos y se ven de inmediato los beneficios del sistema nuevo completo.

ARRANQUE MEDIANTE UN PILOTO

Si más de una unidad empresarial va a utilizar el sistema nuevo, primero se introduce durante un período en una sola unidad, en donde se abordan los problemas y el sistema se pule antes de implementarlo en las demás unidades empresariales.

1.6. Producción y mantenimiento

La operación de sistemas involucra todos los aspectos de usar el sistema nuevo o el modificado en todo tipo de condiciones operativas. Obtener lo mejor de uno u otro durante su operación es el aspecto más importante de las operaciones de los sistemas para muchas organizaciones.

1.6.1. Producción

La gestión de la configuración es un conjunto de actividades de seguimiento y control que comienzan al iniciarse el proyecto de desarrollo y terminan sólo cuando el sistema deja de estar en funcionamiento. Un proceso de gestión de la configuración trabaja con las pautas de referencia estipuladas en el repositorio del proyecto. Una pauta de referencia es una versión del producto del desarrollo que se revisa y aprueba formalmente. Cualquier cambio a la pauta de referencia requiere procedimientos formales de control de cambios. (Beynon-Davies, 2014)

Administración del cambio	• Esto implica hacer un seguimiento de las peticiones de cambios al software por parte de clientes y desarrolladores, estimar los costos y el efecto de realizar dichos cambios.
Gestión de versiones	• Esto incluye hacer un seguimiento de las numerosas versiones de los componentes del sistema y garantizar que los cambios hechos por diferentes desarrolladores a los componentes no interfieran.
Gestión de la configuración	• Matener actualizada las versiones de hardware y software necesarios para que el sistema de información.
Gestión de entregas	• Esto implica preparar el software para la entrega externa y hacer un seguimiento de las versiones del sistema que se entregaron para uso del cliente.

Gráfica 20: ACTIVIDADES EN LA ETAPA DE IMPLEMENTACIÓN **FUENTE:** (Pressman, 2006)

1.6.2. Mantenimiento

La flexibilidad de los sistemas de software es una de las razones principales por las que cada vez más software se incorpora en los sistemas grandes y complejos. Una vez tomada la decisión de fabricar hardware, resulta muy costoso hacer cambios a su diseño. Sin embargo, en cualquier momento durante o después del desarrollo del sistema, pueden hacerse cambios al software. Incluso los cambios mayores son todavía más baratos que los correspondientes cambios al hardware del sistema. (Sommerville, 2011)

El desarrollo del software no se detiene cuando un sistema se entrega, sino que continúa a lo largo de la vida de éste. Después de distribuir un sistema, inevitablemente debe modificarse, con la finalidad de mantenerlo útil. Tanto los cambios empresariales como los de las expectativas del usuario generan nuevos requerimientos para el software existente. Es posible que tengan que modificarse partes del software para corregir errores encontrados durante su operación, para adaptarlo a los cambios en su plataforma de software y hardware, y para mejorar su rendimiento u otras características no funcionales.

Mantenimiento Correctivo	Mantenimiento perfectivo	Mantenimiento adaptativo
Se necesita corregir una falla, puede ser de análisis, de diseño, en el código, en la documentación o cualquier tipo de falla.	Se hace un cambio al código para mejorar la efectividad del producto.	Se hace un cambio al producto para reaccionar a un cambio dentro del entorno en el cual opera el producto.

Gráfica 21: TIPOS DE MANTENIMIENTO EN UN SISTEMA DE INFORMACION **FUENTE:** (Pressman, 2006)

Ciclo de vida de los sistemas de información

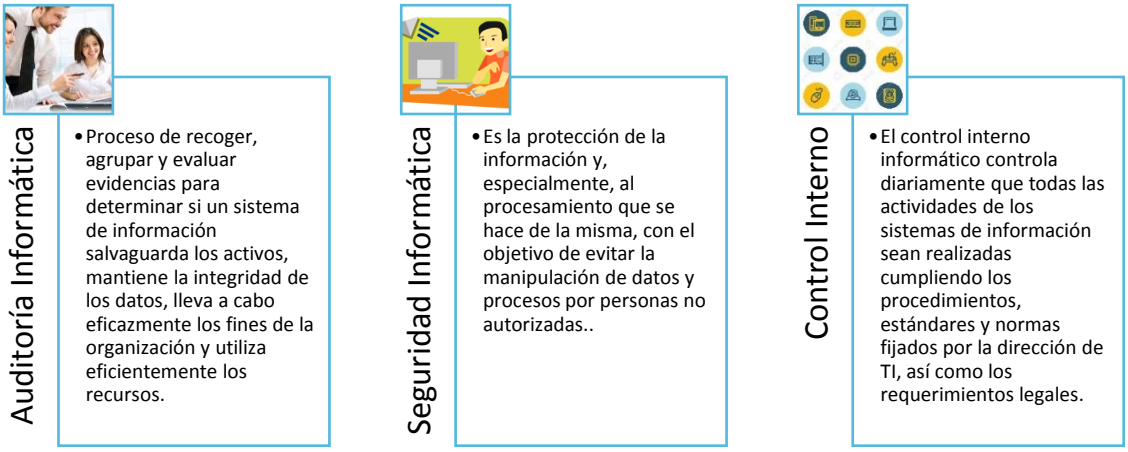
Durante el ciclo de vida del software, se invierte más tiempo en el mantenimiento posentrega que en cualquier otra actividad. El mantenimiento posentrega es el más difícil de todos los aspectos de la producción de software. Una de las principales razones es que incorpora aspectos del resto de los flujos de trabajo del proceso de software.

El mantenimiento posentrega puede verse como el servicio posterior a la venta. El producto se ha entregado al cliente. Pero el cliente está insatisfecho, ya que el producto no trabajo de forma correcta, no hace todo lo que el cliente quiere que haga, o las circunstancias para las cuales se construyó el producto han cambiado de alguna forma. A menos que la organización de software proporcione un buen servicio de mantenimiento, el cliente llevará cualquier producto de desarrollo futuro a otra parte.

El proceso de mantenimiento puede ser especialmente difícil para el software más antiguo. Un sistema heredado es un sistema antiguo que puede estar parchado o modificado repetidamente a lo largo del tiempo.

1.7 Evaluación del Sistema

Con frecuencia, la revisión de los sistemas involucra su monitorización, lo que se llama medición de desempeño del sistema. El número de errores encontrados, la cantidad de memoria requerida, la cantidad necesaria de tiempo de procesamiento o de cpu, y otros problemas deben observarse estrechamente. Si un sistema particular no se desempeña como se espera, debe modificarse, desarrollarse o adquirirse un nuevo sistema. Los productos de desempeño del sistema se desarrollan para medir todos sus componentes, incluidos hardware, software, bases de datos, telecomunicaciones y sistemas de red.



Gráfica 22: ACTIVIDADES PARA EVALUAR SISTEMA DE INFORMACIÓN **Fuente:** PRIETO P., 2022

1.7.1 Auditoría Informática

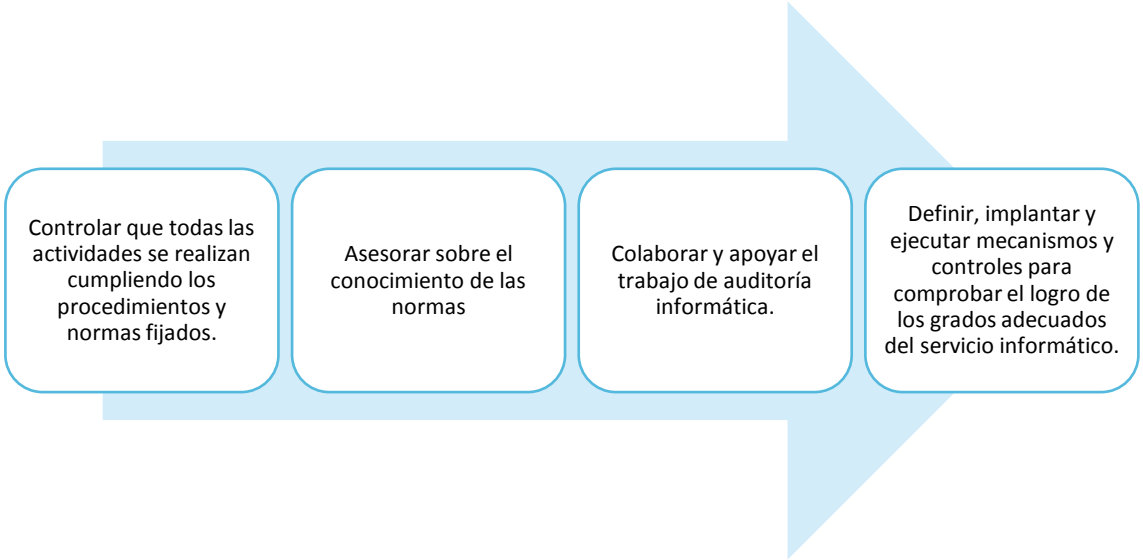
Tras un periodo de operación se debe llevar a cabo una auditoría post-implementación del sistema. Kumar (1990) informó sobre un estudio empírico estadounidense de la prevalencia y forma de evaluación de los sistemas de información tras su implementación. De estos datos, tres son los resultados principales: • La razón principal para realizar una evaluación post-implementación es la formalización del cierre del proyecto de desarrollo. La auditoría es así una

de las principales tácticas de la estrategia de desvinculación del proyecto. • Gran parte de la evaluación la gestionan y la realizan aquellos que han diseñado el sistema implementado.

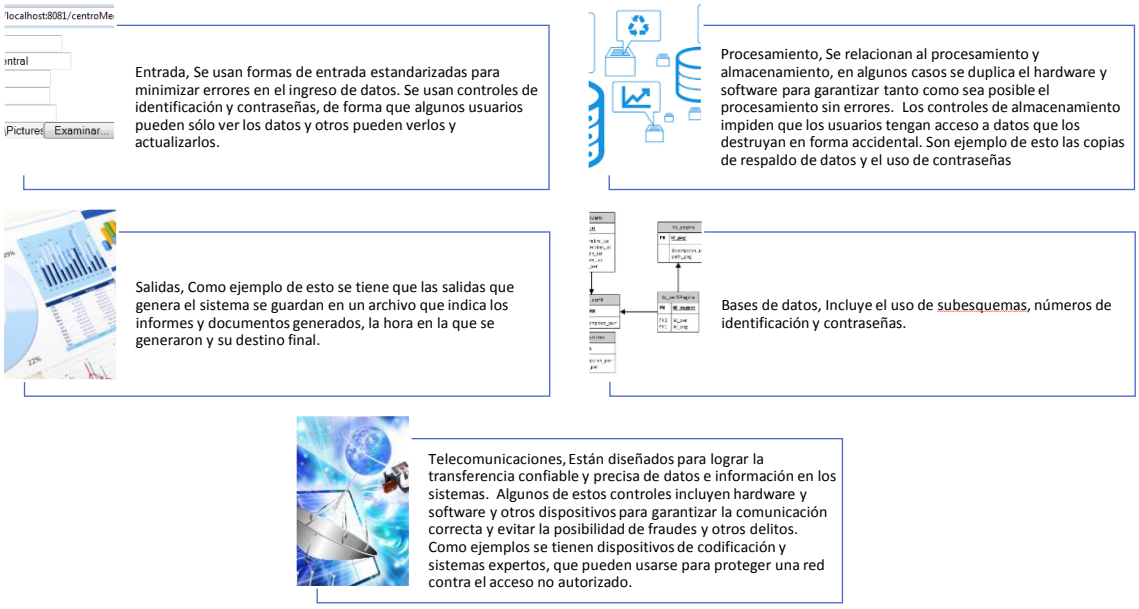
La auditoría informática es el proceso de recoger, agrupar y evaluar evidencias para determinar si un sistema de información salvaguarda los activos, mantiene la integridad de los datos, lleva a cabo eficazmente los fines de la organización y utiliza eficientemente los recursos.

1.7.2 Controles Internos

El control interno informático controla diariamente que todas las actividades de los sistemas de información sean realizadas cumpliendo los procedimientos, estándares y normas fijados por la dirección de TI, así como los requerimientos legales.



Gráfica 23: ACTIVIDADES PARA IMPLANTAR CONTROLES INTERNOS **Fuente:** (Pressman, 2006)



Gráfica 24: EJEMPLOS DE CONTROLES INTERNOS **Fuente:** (Pressman, 2006)

Bibliografía

- https://standards.ieee.org/ieee/830/1222/
- https://www.iso.org/obp/ui/#iso:std:iso-iec-ieee:29148:ed-2:v1:en
- Beynon-Davies, P. (2014). *Sistemas de información : Introducción a la Informática en las organizaciones.* ProQuest Ebook Central: Editorial Reverté.
- Pressman, R. S. (2006). *Ingeniería del Software un enfoque práctico.* México: McGraw Hill.
- Sommerville, I. (2011). *Ingeniería de Software.* México: Pearson.
- Stair, R., & Reynolds, G. W. (2010). *Principios de Sistemas de Información: enfoque administrativo.* México: CENGAGE-Learning.